数组名不是指针：一个流传最广的 C 语言误解

Benjamin 2026-10-09 公众号文章 · 原链接 mp.weixin.qq.com

先看一段代码。不运行，先猜输出：

```c
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 25, -3, 42, 7};
    int *p = a;

    printf("%zu\n", sizeof(a));    /* ? */
    printf("%zu\n", sizeof(p));    /* ? */
    return 0;
}
```

如果"数组名就是指针"这句话成立，`a` 和 `p` 同为 `int *`，两行输出应当相同。但在 64 位机器上，实际输出是 20 和 8。

这不是编译器的怪癖。数组名确实不是指针，它只是在大多数表达式里表现得像指针。这个"表现得像"养出了 C 语言里流传最广的误解，也埋下了不少真实的线上 bug。这篇文章把这件事一次讲清楚。

一、三个铁证

铁证一：sizeof 的答案不同。 20 是 5 个 `int` 的总大小；8 是一个指针的大小（32 位机器上是 4）。如果 `a` 真是指针，它自己只占 8 字节，`sizeof` 不可能知道它背后还挂着 20 字节的数组。

铁证二：数组名不能自增。

```c
int a[5];
int *p = a;

p++; /* OK：p 是变量，可以改 */
a++; /* 编译错误 */
```

gcc 的报错很直白：`error: lvalue required as increment operand`。标准层面说得更直白：C11 §6.3.2.1 在定义"可修改左值"时，明确排除了数组类型。自增和整体赋值要求的都是一个"能被写入的位置"，而数组名给不出这个位置。这也解释了 `a = b;` 为什么非法：C 里能整体赋值的只有结构体和联合体，数组不行，它的内存是整体分配的连续空间，却没给"整体写入"留语义。

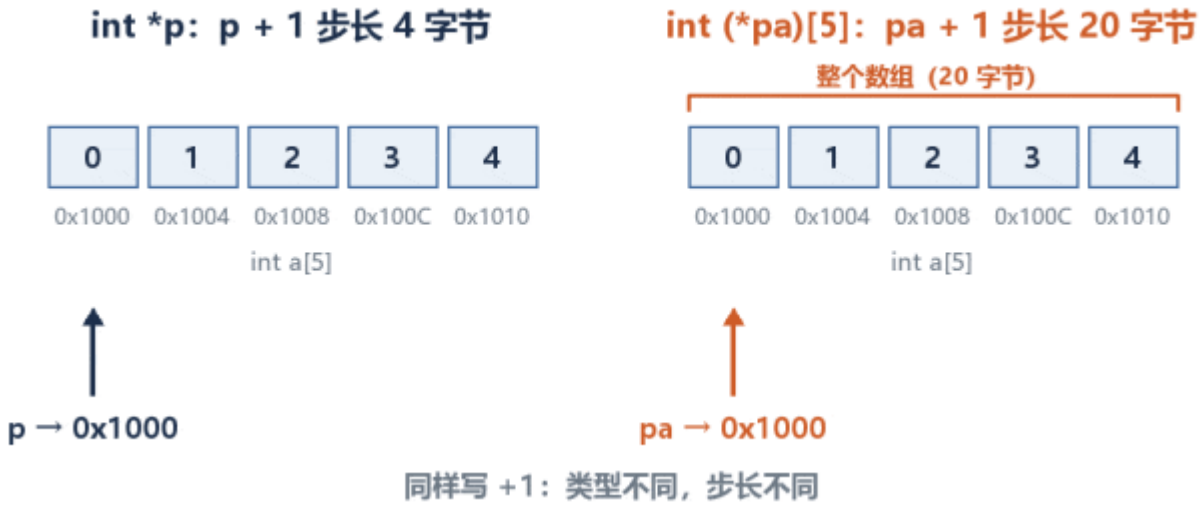
铁证三：`a` 与 `&a` 数值相同，类型不同。

```c
int a[5];
int *pa1 = a;         /* a 退化为 int*   */
int (*pa2)[5] = &a;   /* &a 是 int(*)[5]  */

printf("%td\n", (char *)(pa1 + 1) - (char *)pa1);   /* 4 */
printf("%td\n", (char *)(pa2 + 1) - (char *)pa2);   /* 20 */
```

a 与 &a 打出来数值相同，但上面这组步长实验已经把它们彻底分开：步长差了 5 倍：`pa1 + 1` 跳过一个 `int`，`pa2 + 1` 跳过整个数组。类型决定步长——"数组名是指针"这句话混淆的，正是"值"和"类型"这两件事。

还有个更细的问题：为什么 `a` 与 `&a` 数值**必然**相等？因为数组对象的首地址，按定义就是它第一个字节的地址，而首元素恰好从那里开始。相等是内存布局的必然，不是巧合。相等的是数值，不相等的是类型与步长。



动图：同样写 `+1`，`int *` 一步步走，`int(*)[5]` 一步跳过整个数组。

二、数组名的两重身份

C 标准（C11 §6.3.2.1）把规则写得很清楚：

> 除作为 `sizeof` 、 `_Alignof` 和取地址 `&` 的操作数，以及用于初始化数组的字符串字面量之外，"数组类型的表达式"会被转换为"指向首元素的指针"。

翻译一下：数组名有两重身份。

- 身份一：在大多数表达式中，数组名会退化（decay）为指向首元素的指针值。注意，是值，不是变量：退化产物是个右值，没有存储空间，所以不能被赋值、不能自增。
- 身份二：在 `sizeof`、&、字符串字面量初始化这三种常用场合，数组名保持"数组类型"的本来面目。`sizeof` 能给出整个数组的大小、&a 的类型是 `int(*)[5]`，都靠这一条。

这里还藏着一个细节：`sizeof` 是**编译期**运算符，不是函数。编译器在编译期就掌握着 `int a[5]` 的完整类型信息，直接把常数 20 写进指令。数组名在 `sizeof` 里"不退化"的底气，正来自这份类型信息。另一个条款（§6.5.3.4）还规定：`sizeof` 的操作数不求值，`sizeof(i++)` 不会真的执行 `i++` 。唯一的例外是 C99 引入的变长数组（VLA，C11 起改为可选特性）：`int vla[n]` 的长度只有运行时才知道，它的 `sizeof` 只能推迟到运行时求值。

两种身份的差异可以压成一张表：

	`int a[5]`	`int *p = a`
自身存储	就是那 20 字节元素本身	额外占 8 字节存地址
`sizeof`	20（整个数组）	8（指针）
能否 ++、被赋值	不能	能
`x + 1` 的步长	退化后按 `int`，4 字节	按 `int`，4 字节
`&x` 的类型	`int(*)[5]`，步长 20	`int **`，步长 8

"自身存储"一行是本质，其余都是它的推论：指针变量是一个实实在在的对象，有自己的地址、自己的 8 字节；数组名只是一个名字，它的存储就是数组元素本身。说"数组名就是指针"，类似于说"门牌号就是那栋房子"——门牌号只是个记号，房子才是实体。



图：`a` 、`&a` 、`&a+1` 落在内存布局上的真实含义。地址相同，类型与步长不同。

顺带把下标访问也统一掉：无论 `x` 是数组名还是指针，`x[i]` 都严格等价于 `*(x + i)` 。两者的差别不在 `x[i]` 怎么算，而在 `x` 的"步长类型"从哪来：数组名来自元素类型，指针来自声明类型。数组与指针经常穿着相似的衣服（都能下标访问），但衣服底下是两种东西。这也是为什么上面铁证三里，`pa1` 与 `pa2` 能用同样的语法做加减，步长却完全不同：语法相同，类型不同。

三、误解从哪来：函数参数

误解如此流行，不是没有原因的。把数组传给函数，你会看到"数组名就是指针"的行为证据：

```c
void func(int a[])              /* 实际按 void func(int *a) 处理 */
{
    printf("%zu\n", sizeof(a));  /* 8，不是 20 */
}

int main(void)
{
    int a[5];
    func(a);
    return 0;
}
```

这不是数组名"进了函数就变成指针"，而是 C 标准的另一条规定（C11 §6.7.6.3）：**参数声明中的"数组类型"一律被调整为"指向元素的指针"**。`int a[]` 写在参数位置，从进入函数那一刻起就是 `int *a` ，跟数组名的本性无关。

这条规则是误解最大的温床：在函数里，数组名的行为确实和指针完全一致。但它只说明"传参退化"，不能反推"数组名是指针"。

由此带来一个经典的隐蔽 bug：在函数里求数组长度。

```
#define ARRAY_LEN(a)  (sizeof(a) / sizeof((a)[0]))

void func(int a[])
{
    for (size_t i = 0; i < ARRAY_LEN(a); i++)   /* 循环 2 次，不是 5 次 */
        process(a[i]);
}
```

64 位下 `sizeof(a) / sizeof(a[0])` 等于 8 / 4 = 2，编译器一声不吭，循环次数悄悄减半。可靠的做法只有一个：把长度作为参数显式传入，即 `void func(int *a, size_t n)`。

有人试图"修补"这个宏：既然传进来的是指针，那就改成 `sizeof(p) / sizeof(p[0])`？64 位下 8 / 4 还是 2。只要手里握着的是指针，这个套路就给不出长度。

这个坑严重到 Linux 内核直接在宏里设了编译期防线。内核的 `ARRAY_SIZE` 长这样：

```
#define ARRAY_SIZE(arr) \
    (sizeof(arr) / sizeof((arr)[0]) + __must_be_array(arr))

#define __must_be_array(a) \
    BUILD_BUG_ON_ZERO(__same_type((a), &(a)[0]))
```

关键在 `__same_type((a), &(a)[0])`：若 `a` 是真数组 `int[5]`，它与 `&(a)[0]`（类型 `int *`）类型不同，加数为 0，一切照常；一旦有人把指针传进来，两个表达式类型相同，`BUILD_BUG_ON_ZERO` 就会构造出一个"负位域宽度"，编译当场失败。误用撑不到线上，编译期就爆掉。

这道防线依赖 GNU 扩展 `typeof` 的一个特性：对数组表达式求类型时不退化，`typeof(a)` 得到的就是 `int[5]` 本身。C23 已把 `typeof` 收编为标准关键字，判断"手里到底是数组还是指针"，从此有了标准的写法。

这个等价关系本身也有历史来源。Ritchie 在《The Development of the C Language》里记载：C 的前身 B 语言为了实现简单，数组在表达式中一律按地址处理；C 保留了这个等价，但补上了 `sizeof` 与 `&` 两个例外，让数组保住了"实体"的身份。教科书把"退化"简化成"数组名就是首地址"，丢掉的就是这两个例外——而工程上的坑，几乎全藏在这两个例外里。

顺着这段历史，还能回答一个常见追问：为什么结构体可以整体赋值，数组不行？结构体的整体赋值（语义为逐成员拷贝）从 C89 起就是语言特性；而数组的演进走的是"表达式中按地址处理"这条路，整体赋值的语义从未补上。这是历史设计，不是技术必然。理解了这一层，就不会再对"数组不能赋值、必须 memcpy"感到别扭，也更不会写出 `a = b;` 还纳闷编译器为什么报错。

写在最后

把这篇的结论压成五条，可以当检查表用：

1. 只要 a 在当前位置**仍是数组类型**而不是指针，`sizeof(a)/sizeof(a[0])` 就能求出长度；数组一旦传参退化为指针，这个套路立刻失效，长度必须单独传。要防呆，照抄内核 ARRAY_SIZE 的 `__must_be_array` 写法，让误传指针在编译期报错。
2. 数组名不是可修改的左值：不要写 `a++`、`a = b`。需要"整体赋值"语义，用结构体，或 memcpy。
3. a 与 &a 数值相同、类型不同。不要为了"统一类型"把 &a 强转回 `int *`，步长信息一强转就丢。
4. 记住三种常用的不退化场合：`sizeof`、取地址 &、字符串字面量初始化（标准里还有个 `_Alignof`，工程中极少见）。另外，`char a[] = "hi"` 得到的是可修改的数组副本，`char *s = "hi"` 得到的是指向字面量的指针。这两行代码天差地别，值得单独一篇。
5. 二维数组退化只降第一维：`int a[3][4]` 传参后是 `int (*a)[4]`，列数决定步长，不能写成 `int **`。

下一篇我们顺着 `int (*pa)[5]` 这个类型往下走：为什么有人看得懂 `int *p[10]`，却看不懂 `int (*p)[10]`？我会给出一套拆解任意复杂声明的方法，连 `signal` 的原型也能三步读完。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。